

Figure 1: Buddy allocator used for physical page allocation. Already allocated pages are shaded, while free pages are tracked by the free lists.

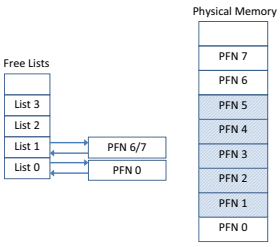


Figure 2: Buddy allocator state after an allocation for 2 pages is finished.

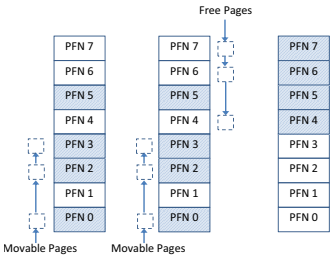


Figure 3: The memory compaction daemon tracks movable and free memory pages, exchanging them to eliminate fragmentation.

application makes a `malloc` call, simultaneously requesting N physical pages from the OS. The buddy allocator first searches the free list entry corresponding to the smallest contiguous page frames bigger than N (entry $\lceil log_2(N) \rceil$). If a block of free physical pages is found in that list, allocation successfully completes. Otherwise, the free list is progressively climbed until an entry with a block of free contiguous physical pages is found. Once a free block is found, the buddy allocator must minimize memory fragmentation. Therefore it iteratively halves the block, inserting these new blocks in their appropriate free list locations, until it extracts a block of N contiguous physical pages. As an example, Figure 2 shows the state of the free list after an application level request for two physical pages to be allocated. At first, entry 1 in the free list is checked; however, since this is empty, entry 2 is scanned. Here, a free block with contiguous physical pages 4, 5, 6, and 7 is found. Hence, the buddy allocator halves this block of four pages, returning pages 4 and 5 to the application and moving pages 6 and 7 to free list entry 1. Apart from allocation, the buddy allocator also updates its state when physical pages are released. At this point, the kernel attempts to merge pairs of free *buddy* blocks if both have the same size and are contiguous.

By construction therefore, the buddy allocator deliberately provides contiguous physical page frames to the application when it asks for multiple page frames together. Since applications usually make `malloc` calls that simultaneously request a number of physical pages together (rather than one page at a time), the buddy allocator is able to provide them a suitable contiguous range of pages. We will show that the buddy allocator successfully produces this contiguity even in the presence of significant system load. While insufficient for superpages, CoLT benefits from this substantially.

3.2.2. Memory compaction. In order to glean contiguous runs of physical pages, the buddy allocator relies on ensuring that memory fragmentation is tightly controlled. However, fragmentation is pronounced when multiple processes with large working sets simultaneously run on the system. Therefore, many operating systems boost the buddy allocator with a separate memory compaction daemon. Figure 3 details the Linux memory compaction daemon in three steps on a heavily-fragmented system.

First, as shown in the left-most diagram of Figure 3, memory compaction runs an algorithm that starts at the bottom of the physical memory and builds a list of allocated pages that are *movable*. While most user-level pages are movable, pinned and kernel pages usually are not. Nevertheless, user-level pages usually outnumber kernel pages, making most pages movable.

Second, the daemon starts at the top of physical memory and builds a list of free pages. Eventually, the two algorithms meet in the middle of the physical page list. At this point, Linux invokes migration code to shift the movable pages to the free page list, yielding the unfragmented diagram at the right of Figure 3.

Since there is a cost associated with moving pages, the compaction daemon is only triggered when there is heavy system fragmentation. As such, its operation naturally produces contiguity, especially in tandem with the buddy allocator. In fact, we will show that this daemon successfully generates contiguity even under heavy system fragmentation.

3.2.3. Transparent hugepage support. Aside from buddy allocation and memory compaction, support for superpages is a primary cause of page allocation contiguity. Unfortunately superpage management comes with high overheads. As a result, Linux's Transparent Hugepage Support (THS), supported since the 2.6.38 kernel [4], uses superpages sparingly. When THS is enabled, the memory allocator attempts to find a free 2MB block of memory. If this block is naturally aligned at a 2MB boundary, a superpage is constructed. In practice, the OS relies on the memory compaction daemon to construct these 2MB regions. Aligned 2MB regions are rare; when a superpage cannot be constructed, the system defaults to the buddy allocator. Even when the 2MB pages are allocated, increased load can eventually make them harmful. In these cases, system pressure triggers a daemon that breaks superpages into baseline 4KB pages.

In practice, THS struggles to maintain many superpages simultaneously. However, it does succeed in creating additional levels of contiguity for two reasons. First, while optimistically-allocated 2MB superpages are often eventually split due to system pressure, they retain contiguity among tens of baseline 4KB pages. Second, THS relies on the memory compaction daemon, triggering it more often and providing the buddy allocator even higher levels of contiguity suitable for CoLT.

3.2.4. System Load and Memory Fragmentation. Finally, page allocation contiguity is deeply affected by the system load. If many processes run simultaneously, main memory is likelier to be fragmented. Therefore, one may initially expect that higher load degrades contiguity. Surprisingly, we will show that contiguity can actually *increase* with greater system load. This occurs because system load has a complex relationship with the memory compaction daemon, triggering it more often when there is higher load. This can, in turn, free up more contiguous physical frames for the buddy allocator, eventually resulting in more contiguity.

4. CoLT Design and Implementation

Having detailed contiguity sources, we now propose three variants of CoLT. Overall, they share three design principles. First, they detect instances of consecutive virtual-to-physical address translations. These entries are coalesced into single TLB entries, so as to reduce miss rates. Second, CoLT coalesces only on *TLB misses*. While TLB hits could also prompt coalescing, this may increase